A

Project Report

Submitted in partial fulfilment of the requirements for the completion of the Course

EDS Lab Codetantra

Practical No.2

(2304102L)

Submitted by,

Harshvardhan Lokhande (202501090156)

Under the Guidance of,

Ms. Savita Mane(SM)

MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Department of Mechanical Engineering,

MIT Academy of Engineering,

Alandi road, Pune - 412 105,

Maharashtra (India)

2.1.1. List operations

08:32

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add

2. Remove

3. Display

4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

> **Add**: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
>
> **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
>
> **Display** : Displays the current list of integers. If the list is empty, display "List is empty".
>
> **Quit**: Exits the program.
>
> The program should handle invalid menu choices by displaying "Invalid choice".Ensure that the program continues to prompt the user until they choose to quit (option 4).

listOps.py

```
int_list = []

while True:
    print("1.
    Add")
    print("2. Remove")
    print("3. Display")
    print("4. Quit")

    choice = input("Enter choice: ")

    if choice == '1': # Add
        value = input("Integer: ")
        try:
            num = int(value)
            int_list.append(num)
            print("List after adding:", int_list)
        except ValueError:
            print("Invalid input")

    elif choice == '2': #
        Remove if not int_list:
            print("List is empty")
        else:
            value = input("Integer: ")
            try:
```

```
num = int(value)
```

```
            if num in int_list:
                int_list.remove(num)
                print("List after removing:", int_list)
            else:
                print("Element not found")
        except ValueError:
            print("Invalid input")

elif choice == '3': #
    Display if not int_list:
        print("List is empty")
    else:
        print(int_list)

elif choice == '4': # Quit
    break

else:
    print("Invalid choice")
```

Explorer

Average time
0.037 s
37.33 ms

Maximum time
0.056 s
56.00 ms

2 out of 2 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Debugger

Test case 1 30 ms Debug

Expected output	Actual output
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 1	Enter choice: 1
Integer: 11	Integer: 11
List after adding: [11]	List after adding: [11]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 1	Enter choice: 1
Integer: 25	Integer: 25
List after adding: [11, 25]	List after adding: [11, 25]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 2	Enter choice: 2
Integer: 26	Integer: 26
Element not found	Element not found
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 2	Enter choice: 2
Integer: 11	Integer: 11
List after removing: [25]	List after removing: [25]
1. Add	1. Add
2. Remove	2. Remove

Terminal Test cases

‹ Prev Reset Submit Next ›

Explorer

Average time
0.072 s
71.67 ms

Maximum time
0.089 s
89.00 ms

2 out of 2 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Debugger

Integer: 11	Integer: 11
List after removing: [25]	List after removing: [25]
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 2	Enter choice: 2
Integer: 25	Integer: 25
List after removing: []	List after removing: []
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 3	Enter choice: 3
List is empty	List is empty
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 2	Enter choice: 2
List is empty	List is empty
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 8	Enter choice: 8
Invalid choice	Invalid choice
1. Add	1. Add
2. Remove	2. Remove

‹ Prev Reset Submit Next ›

List is empty	List is empty
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 8	Enter choice: 8
Invalid choice	Invalid choice
1. Add	1. Add
2. Remove	2. Remove
3. Display	3. Display
4. Quit	4. Quit
Enter choice: 4	Enter choice: 4

‹ Prev Reset Submit Next ›

2.1.2. Dictionary Operations

07:23

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

All operations must be performed using dictionary methods.

Perform deletion only if possible; leave the dictionary unchanged.

Refer to the visible test cases for better understanding and strictly match with the input/outputs.

Initial dictionary with 10 predefined records

student = {

```
    1: "Amit",
    2: "Riya",
    3: "Kiran",
    4: "Neha",
    5: "Arjun",
    6: "Pooja",
    7: "Rahul",
    8: "Sneha",
    9: "Vikram",
    10: "Anjali"
}

# Display original dictionary
print("Original Dictionary:", student)

# Insertion
key = int(input())
value = input()
student[key] =
value
print("After Insertion:", student)

# Update
upd_key = int(input())
upd_value = input()
if upd_key in student:
        student[upd_key] =
```

upd_value print("After Update:",

student)

```
# Deletion
del_key = int(input())
if del_key in student:
        student.pop(del_key)
print("After Deletion:",
student)

# Traversal
print("Traversing Dictionary:")
for k, v in student.items():
        print(k, ":", v)
```

Average time 0.027 s 27.00 ms

Maximum time 0.044 s 44.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Expected output	Actual output
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}	Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
11	11
Suresh	Suresh
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}	After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
3	3
Karthik	Karthik
After Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}	After Update: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
5	5
After Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}	After Deletion: {1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'}
Traversing Dictionary:	Traversing Dictionary:
1 : Amit	1 : Amit
2 : Riya	2 : Riya
3 : Karthik	3 : Karthik
4 : Neha	4 : Neha
6 : Pooja	6 : Pooja
7 : Rahul	7 : Rahul
8 : Sneha	8 : Sneha
9 : Vikram	9 : Vikram
10 : Anjali	10 : Anjali
11 : Suresh	11 : Suresh

Terminal Test cases

< Prev Reset Submit Next >

2.2.1. Linear search

Technique 04:45

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

The first line of input contains the array of integers which are separated by space The last line of input contains the key element to be searched

Output format:

If the element is found, print the index.

If the element is not found, print **Not found** .

Sample Test Case: Input:

1 2 3 4 3 5 6

3

Output:

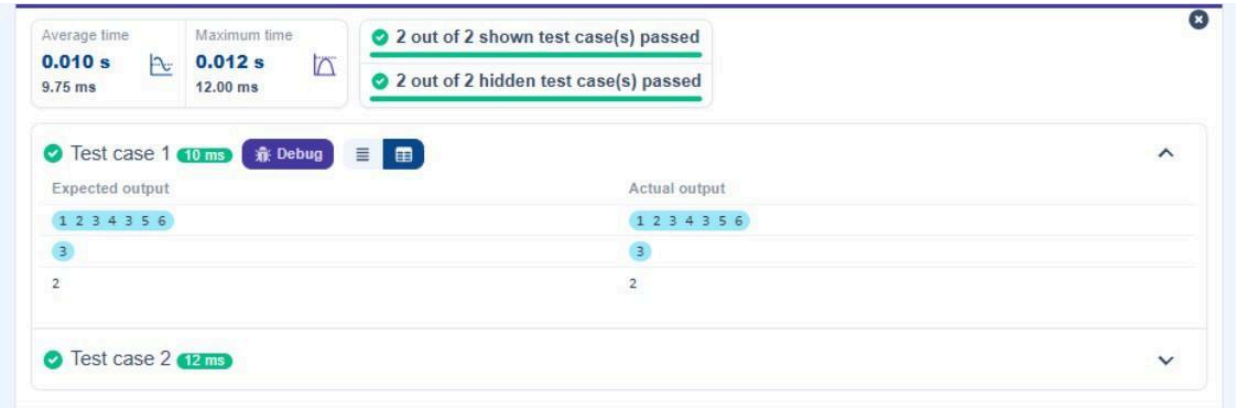
2

```
# Read input array
arr = list(map(int, input().split()))

# Read key element
key = int(input())

# Linear search
found = False
for i in range(len(arr)):
    if arr[i] == key:
        print(i)
        found = True
        break

# If not found
if not found:
    print("Not found")
```



2.2.2. Captain of the

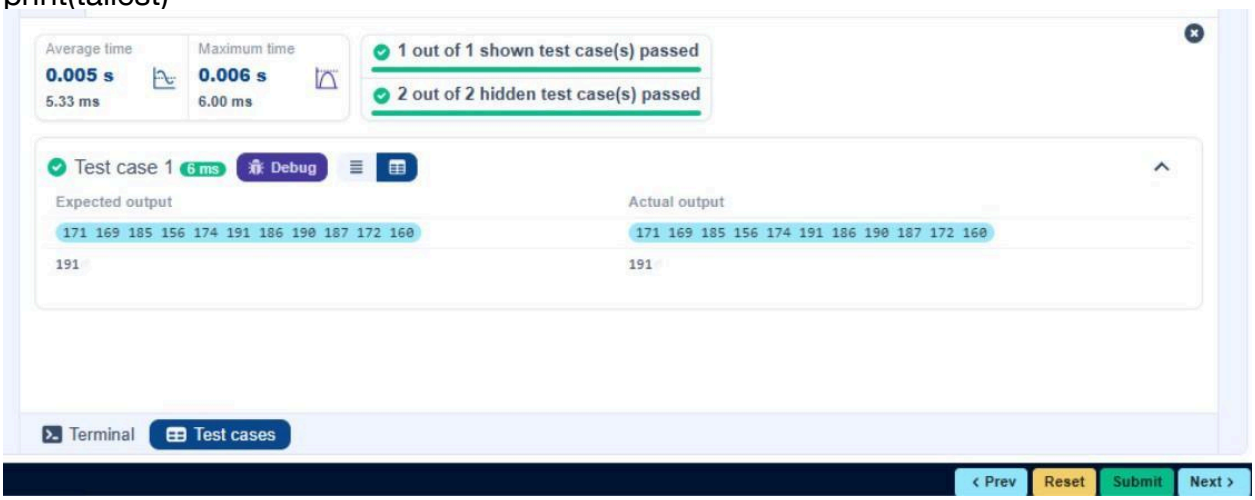
Team 01:32

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format

The output should be the height (in centimeters) of the tallest player.

```
# Read heights of 11 players
heights = list(map(int, input().split()))

# Find the tallest player
tallest = max(heights)

# Output the tallest height
print(tallest)
```